

Autonomous Agentic Workflow Runtime

Complete Integration Architecture

LangChain Deep Agents

OpenCode

Langflow

OpenSandbox

Protocols: MCP · ACP · HTTP/REST · SSE · JSON-RPC · LSP

WHAT THIS DOCUMENT COVERS

- Complete 4-layer integration architecture with OpenSandbox as execution sandbox
- MCP Protocol Bridge wiring all four tools through a universal bus
- 6 enterprise use-case pipelines: Auto Dev · DevOps · Observability · KB · Security · Perf
- 12 Q&A sections with exact API endpoints, plugin code, and config snippets
- Multi-agent coordination: Fork-Join · Supervisor-Worker · Reactive Event Chain
- Enterprise deployment: fleet config, circuit breaker, secrets, RBAC, multi-region
- Reference file/folder structure, AGENTS.md, SKILL.md library, CI/CD bootstrap
- Top 10 failure modes with automated mitigations and human escalation paths

Table of Contents

- 1 Executive Summary
- 2 Tool Inventory & Capabilities
 - 2.1 LangChain Deep Agents SDK
 - 2.2 OpenCode
 - 2.3 Langflow
 - 2.4 OpenSandbox [NEW]
- 3 Layered Architecture (Q1)
- 4 MCP Protocol Bridge — Universal Bus (Q2)
- 5 Agentic Task Decomposition Pipeline (Q3)
- 6 OpenCode Plugin: Deep Agents Bridge (Q4)
- 7 Langflow Flow: Auto App Development (Q5)
- 8 Deep Agents as the Planning Brain (Q6)
- 9 SKILL.md Living Knowledge System (Q7)
- 10 Auto Reactive Observability Pipeline (Q8)
- 11 Multi-Agent Coordination Patterns (Q9)
 - 11.1 Fork-Join Pattern
 - 11.2 Supervisor-Worker Pattern
 - 11.3 Reactive Event Chain
- 12 Enterprise Deployment Architecture (Q10)
- 13 Reference Implementation Blueprint (Q11)
- 14 Failure Modes, Risks & Mitigations (Q12)
- 15 Implementation Roadmap
- 16 Capability Gaps & Open Questions

1. Executive Summary

The **Autonomous Agentic Workflow Runtime (AAWR)** is a production-grade enterprise platform that unifies four best-in-class agentic tools into a single, self-driving engine capable of autonomously delivering software, operating CI/CD pipelines, responding to production incidents, maintaining documentation, patching security vulnerabilities, and optimizing performance — all with full auditability and defined human override paths.

Four-Layer Stack

Layer	Tool	Responsibilities
Orchestration	Langflow	DAG-based flow composition · webhook triggers · LLM planner nodes · human approval gates · MCP server & client · RAG / vector stores · result aggregation
Planning	LangChain Deep Agents / LangGraph	Long-horizon task planning · write_todos tracker · subagent spawning · cross-session memory (LangGraph Memory Store) · HITL interrupts · durable execution
Code Execution	OpenCode	LSP-aware code generation & editing · 30+ language servers · bash / grep / glob / lsp tools · session fork/diff · plugin event hooks · SKILL.md resolver · worktree management
Safe Execution Sandbox	OpenSandbox [NEW]	Isolated container runtime for AI-generated code · stateful CodeInterpreter (Jupyter-backed) · multi-language execution (Python, Go, Java, TS, Rust, Bash) · resource quotas · K8s BatchSandbox pooling

These four layers are unified by **MCP (Model Context Protocol)** as a universal integration bus. Langflow exposes flows as MCP tools; OpenCode consumes MCP tool sources; Deep Agents wraps any Python callable; OpenSandbox exposes its execution API as an MCP server. Every capability is reachable from every other component without point-to-point wiring.

Why OpenSandbox fills a genuine gap: The baseline AAWR used OpenCode's *bash* tool with permission: ask as a safety workaround for test execution. OpenSandbox replaces this with proper container isolation — AI-generated code runs in a throwaway sandbox, never on the host filesystem. It additionally provides stateful CodeInterpreter sessions for the Deep Agent's iterative profiling and data-analysis tasks, and Kubernetes BatchSandbox pooling for sub-second session startup in the enterprise fleet.

Six Enterprise Use-Cases

Auto App Development	GitHub issue → spec → OpenCode implement → OpenSandbox test → PR
Auto DevOps	Merge event → CI flow → build → OpenSandbox test → deploy staging → promote/rollback
Auto Reactive Observability	PagerDuty alert → triage → Deep Agent RCA → OpenCode fix → staging validation → post-mortem
Auto Knowledge Base Maintenance	PR merge / ADR commit → Deep Agent extract learnings → update SKILL.md → git commit
Auto Security Patching	CVE detected → patch branch → OpenCode upgrade deps → OpenSandbox security scan → PR

Auto Performance Optimization	Metric regression → OpenCode LSP profile → Deep Agent CodeInterpreter benchmark loop → optimize → validate
-------------------------------	--

2. Tool Inventory & Capabilities

2.1 LangChain Deep Agents SDK (Python)

A standalone Python library built on LangChain core and the **LangGraph runtime** providing durable execution, streaming, HITL interrupts, and cross-thread persistence. This is the planning brain of the AAWR.

Core API	create_deep_agent(tools=[], system_prompt="") → runnable LangGraph agent
Task decomposition	write_todos built-in tool for multi-step planning and progress tracking
Virtual filesystem	ls, read_file, write_file, edit_file with pluggable backends: in-memory, local disk, LangGraph store, sandboxes
Subagent spawning	Built-in task tool with context isolation — one subagent per microservice
Long-term memory	LangGraph Memory Store — cross-thread, cross-session persistence for enterprise knowledge
HITL	Human-in-the-loop interrupts for irreversible actions
Streaming	Real-time token and event streaming
Tracing	LangSmith integration for traces, debugging, and evaluation

2.2 OpenCode (by Anomaly)

Open-source AI coding agent written in **Go** with a rich JS/TS SDK. Runs as a **headless HTTP server** (opencode serve) on port 4096, exposing an OpenAPI 3.1 spec. The TUI is just a client. This is the LSP-aware code execution engine of the AAWR.

Server	Headless HTTP server · OpenAPI 3.1 · Basic Auth · CORS · mDNS discovery
Session API	create, list, prompt, abort, fork, share, revert, summarize, diff
Tools	bash, edit, write, read, grep, glob, list, lsp, skill, webfetch, websearch
LSP Engine	30+ language servers: TypeScript, Python/pyright, Go/gopls, Rust/rust-analyzer, Java/jdtls
SSE Events	session.idle, session.error, tool.execute.before/after, file.edited, message.updated
Protocols	MCP (tool sources) · ACP (subprocess JSON-RPC) · LSP (language diagnostics)
Plugin system	TypeScript hooks: tool.execute.before/after, shell.env, file.edited, session.compacting
Skills	SKILL.md files with YAML frontmatter — reusable instruction sets per agent/project
Config hierarchy	Remote (.well-known/opencode) → Global → Project (opencode.json) → Runtime (env var)
Ecosystem plugins	opencode-worktree · opencode-background-agents · opencode-daytona · opencode-sentry-monitor

2.3 Langflow

Open-source Python-based **visual editor** for building AI application workflows as DAGs. Flows are collections of component nodes — each node is one processing step. This is the orchestration and trigger layer of the AAWR.

Flows	DAGs of component nodes: LLMs, vector stores, embeddings, memory, data loaders, HTTP
API	REST API POST /api/v1/run/{flow_id} for programmatic flow triggering
MCP dual role	Expose flows AS MCP tools; consume external MCP tools IN flows
Agent support	Built-in agent components (LLM + tool selection loop)
Approval gates	Async human-in-the-loop decision components
Tweaks	Runtime parameter overrides without changing flow definitions
Extensibility	Custom Python components, pluggable LLMs and vector stores

2.4 OpenSandbox [NEW LAYER]

Alibaba open-source **universal sandbox platform** designed for AI application scenarios. Provides a complete solution with multi-language SDKs, OpenAPI-specified protocols, and flexible Docker/Kubernetes runtimes. An execd daemon (Go/Beego) is injected into any base container image at runtime — no image modification required. This is the **safe execution sandbox** layer of the AAWR.

Lifecycle API	POST /sandboxes · GET /sandboxes/{id} · pause · resume · renew · endpoint routing
Execution API	POST /code (SSE) · POST /command (SSE) · filesystem CRUD · GET /metrics/watch
CodeInterpreter	Stateful Jupyter-backed execution: Python, Java, JavaScript, TypeScript, Go, Bash
Commands	Foreground + background shell execution with stdout/stderr SSE streaming
Filesystem	upload, download, glob search, chmod, mv, replace — full remote filesystem API
Runtimes	Docker (production-ready, host + bridge networking) · Kubernetes BatchSandbox with pooling
execd injection	Binary injected at container creation — starts Jupyter, then execd, then user endpoint
Resource quotas	Hard CPU, memory, GPU limits enforced at container level
Security	API key auth (lifecycle) · token auth (execution) · network isolation · kata-containers / gVisor
AAWR role	Replaces bash: ask workaround · provides isolated test execution · CodeInterpreter for Deep Agent

3. Layered Architecture (Q1)

The AAWR is structured as four horizontal layers communicating through the MCP bus, with OpenCode's SSE stream and OpenSandbox's SSE metrics providing real-time event telemetry back to upper layers.

Layer	Tool	Sits At	Does NOT Handle
Orchestration	Langflow	Entry points, flow routing, approval gates, result aggregation	Code generation, code execution, memory
Planning	Deep Agents / LangGraph	Task decomposition, subagent management, enterprise memory	Code execution, flow UI, container lifecycle
Code Execution	OpenCode	LSP-aware editing, file manipulation, language tooling	Isolated runtime, Jupyter, container creation
Safe Execution	OpenSandbox	Isolated container execution, stateful CodeInterpreter, resource limits	Code generation, flow logic, memory

Data Flow: Auto App Development

GitHub Issue → **Langflow** (webhook, classify, decompose) → **Deep Agent** (generate spec via RAG + SKILL.md) → **OpenCode** (implement: LSP-guided edits, filesystem writes) → **OpenSandbox** (run tests in isolated container) → Deep Agent (generate test report) → GitHub MCP (open PR with diff + test evidence).

Data Flow: Auto Reactive Observability

PagerDuty alert → **Langflow** (classify severity, query RAG for past incidents, select SKILL.md playbook) → **Deep Agent** (LSP incomingCalls + findReferences trace error chain, query memory store) → **OpenCode** (worktree, implement fix, commit) → **OpenSandbox** (run test suite in isolation) → **Langflow** (staging deploy, 5-min metric window, auto-approve/escalate) → Deep Agent (post-mortem, update SKILL.md, post to Confluence/Slack).

4. MCP Protocol Bridge — Universal Bus (Q2)

MCP (Model Context Protocol) is the single integration bus. Every capability is exposed as a named tool; every tool is callable from every component. The bridge has four arms: Langflow-as-MCP-server, OpenCode-as-MCP-consumer, Deep-Agents-as-MCP-server (via sidecar), and OpenSandbox-as-MCP-server.

Direction	Source	Target	MCP Tool Names	Transport
Langflow → MCP	Langflow REST API	OpenCode · Deep Agents	langflow_run_flow, langflow_approval	SSE
OpenCode → MCP	External servers	OpenCode sessions	github_*, devops_*, security_*, opensand	stdio/SSE
Deep Agents → MCP	deepagents_mcp_server.py	OpenCode · Langflow	deepagent_spawn, deepagent_memory_get/set	stdio
OpenSandbox → MCP	opensandbox-mcp adapter	Deep Agents · Langflow	sandbox_create, sandbox_run_code, sandbox_exec_command	SSE

opencode.json — Full MCP Server Configuration

opencode.json — project-level config

```
{
  "mcp": {
    "servers": {
      "langflow-mcp": {
        "type": "sse",
        "url": "http://langflow-service:7860/api/v1/mcp/sse",
        "env": { "LANGFLOW_API_KEY": "${LANGFLOW_API_KEY}" }
      },
      "deepagents-mcp": {
        "type": "stdio",
        "command": "python",
        "args": ["-m", "aawr.deepagents_mcp_server"],
        "env": { "LANGSMITH_API_KEY": "${LANGSMITH_API_KEY}" }
      },
      "opensandbox-mcp": {
        "type": "sse",
        "url": "http://opensandbox-service:8080/mcp/sse",
        "env": { "OPENSANDBOX_API_KEY": "${OPENSANDBOX_API_KEY}" }
      },
      "github-mcp": {
        "type": "sse",
        "url": "https://api.githubcopilot.com/mcp/",
        "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" }
      },
      "devops-mcp": {
        "type": "stdio",
        "command": "npx",
        "args": ["-y", "@aawr/devops-mcp-server"]
      }
    }
  },
  "permissions": {
    "langflow-mcp_*": "allow",
    "deepagents-mcp_*": "allow",
    "opensandbox-mcp_*": "allow",
    "github-mcp_create_*": "allow",
    "github-mcp_merge_*": "ask",
    "github-mcp_delete_*": "deny",
    "bash": "ask"
  }
}
```



```

    }
  }
}

```

OpenSandbox MCP Tool Schemas

opensandbox MCP tool definitions

```

{
  "tools": [
    {
      "name": "sandbox_create",
      "description": "Create an isolated sandbox container from a base image",
      "inputSchema": {
        "type": "object",
        "properties": {
          "image": { "type": "string", "default": "python:3.11" },
          "cpu": { "type": "number", "description": "CPU cores limit" },
          "memory": { "type": "string", "description": "Memory limit e.g. 512m" },
          "ttl_seconds": { "type": "integer", "default": 300 }
        },
        "required": []
      }
    },
    {
      "name": "sandbox_run_code",
      "description": "Execute code in a sandbox CodeInterpreter session (stateful, Jupyter-backed)",
      "inputSchema": {
        "type": "object",
        "properties": {
          "sandbox_id": { "type": "string" },
          "language": { "type": "string", "enum": ["python", "javascript", "typescript", "go", "java", "bash"] },
          "code": { "type": "string" },
          "context_id": { "type": "string", "description": "Session ID for stateful execution" }
        },
        "required": ["sandbox_id", "language", "code"]
      }
    },
    {
      "name": "sandbox_exec_command",
      "description": "Run a shell command in an isolated sandbox with SSE output streaming",
      "inputSchema": {
        "type": "object",
        "properties": {
          "sandbox_id": { "type": "string" },
          "command": { "type": "string" },
          "working_dir": { "type": "string" }
        },
        "required": ["sandbox_id", "command"]
      }
    }
  ]
}

```

5. Agentic Task Decomposition Pipeline (Q3)

When a task arrives via webhook (Jira ticket, PagerDuty alert, GitHub issue), Langflow decomposes it into typed subtasks and routes each to the appropriate agent in parallel. OpenSandbox is now used for all test execution steps instead of bare bash.

Step	From	To	API Call / Detail
1	External Source	Langflow Webhook	POST /api/v1/webhook/{flow_id} — { ticket_url, description, source }
2	Langflow	LLM Planner	Classify + decompose into typed subtasks: coding / research / devops
3	Langflow	OpenCode HTTP	POST /session { projectPath, agentId } → { sessionId }
4	Langflow	OpenCode HTTP	POST /session/{id}/message { content: spec, role: user }
5	Langflow	OpenCode SSE	GET /event — stream tool.execute.after, file.edited events
6	Langflow	OpenCode HTTP	GET /session/{id}/diff → FileDiff[]
7	Langflow	OpenSandbox MCP	sandbox_create { image: 'node:20', ttl: 300 } → sandbox_id
8	Langflow	OpenSandbox MCP	sandbox_exec_command { command: 'npm test', sandbox_id } — SSE stream
9	Langflow	Deep Agent	deepagent_spawn { task: write tests, context: diffs }
10	Langflow	GitHub MCP	github_create_pull_request { branch, title, body: summary }

Async Polling + OpenSandbox Test Execution

TypeScript — session completion + sandbox test runner

```
async function runPipelineForTask(
  ocClient: OpenCodeClient,
  osClient: OpenSandboxClient,
  projectPath: string,
  spec: string
): Promise<PipelineResult> {

  // 1. Create OpenCode session and submit coding task
  const session = await ocClient.session.create({ projectPath, agentId: "feature-dev" });
  await ocClient.session.prompt(session.id, { content: spec, role: "user" });

  // 2. Stream SSE events until session.idle
  await new Promise<void>((resolve, reject) => {
    const stream = ocClient.events.subscribe({ sessionId: session.id });
    stream.on("session.idle", () => resolve());
    stream.on("session.error", (e: Error) => reject(e));
    setTimeout(() => reject(new Error("timeout")), 300_000);
  });

  // 3. Retrieve file diffs
  const diffs = await ocClient.session.diff(session.id);

  // 4. Create isolated OpenSandbox instance for test execution
  const sandbox = await osClient.sandbox.create({
    image: "node:20-alpine",
    cpu: 2, memory: "1g", ttl_seconds: 300
  });
```

```
// 5. Upload changed files into sandbox
for (const diff of diffs) {
  await osClient.sandbox.filesystem.upload(sandbox.id, diff.path, diff.content);
}

// 6. Run tests in isolation – full blast-radius zero
const testResult = await osClient.sandbox.commands.run(sandbox.id, {
  command: "npm ci && npm test -- --json",
  streaming: true
});

// 7. Destroy sandbox (or let TTL expire)
await osClient.sandbox.destroy(sandbox.id);

return { diffs, testResult };
}
```

6. OpenCode Plugin: Deep Agents Bridge (Q4)

The aawr-deepagents-bridge plugin wires OpenCode's event hooks into the rest of the AAWR. Four hooks are implemented. The destructive-command hook now routes to OpenSandbox for safe re-execution rather than blocking outright.

Hook	Trigger Condition	Action
session.idle	Session completes	Serialize history + FileDiff[] → send to Deep Agent for post-processing (write tests, update SKILL.md, etc)
tool.execute.before [bash]	Bash command matches destructive pattern → blocked by approval gate	Route to sandbox for safe re-execution if approved, re-run in OpenSandbox container instead
experimental.session.compacting	Session context is being compacted	Fetch enterprise context (current sprint, active incidents, team conventions) from Langflow
file.edited [*.infra.tf]	Terraform file modified	Trigger Langflow infrastructure-validation-flow → if valid, spawn Deep Agent security review

```
./opencode/plugins/aawr-deepagents-bridge.ts (key hooks)

import { definePlugin, tool } from "@opencode-ai/plugin";
import { z } from "zod";

const DESTRUCTIVE = [/rm\s+-rf/, /kubectl\s+delete/, /terraform\s+destroy/, /DROP\s+TABLE/i];

export default definePlugin({
  name: "aawr-deepagents-bridge",
  version: "1.1.0",
  hooks: {

    // Hook A: Post-process completed sessions
    "session.idle": async ({ session, client }) => {
      const [messages, diffs] = await Promise.all([
        client.session.messages(session.id),
        client.session.diff(session.id)
      ]);
      await fetch(`${process.env.DEEPAGENTS_MCP_URL}/invoke`, {
        method: "POST", headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ task: "post_process_session",
          context: { sessionId: session.id, conversationHistory: messages, fileDiffs: diffs,
            instructions: "Write unit tests for all edited files. Update SKILL.md entries"
          }
        })
      });
    },

    // Hook B: Intercept destructive bash — route to sandbox instead of host
    "tool.execute.before": async ({ tool: toolName, input, session }) => {
      if (toolName !== "bash") return;
      if (!DESTRUCTIVE.some(p => p.test(input.command))) return;

      // Gate: require Langflow approval
      const approval = await fetch(
        `${process.env.LANGFLOW_URL}/api/v1/run/${process.env.APPROVAL_FLOW_ID}`,
        { method:"POST", headers:{"x-api-key": process.env.LANGFLOW_API_KEY!,
          "Content-Type":"application/json"},
          body: JSON.stringify({ input_value: JSON.stringify({
            command: input.command, sessionId: session.id }) })
        }).then(r => r.json());

      if (approval.outputs[0].outputs[0].results.message.text !== "APPROVED") {
        throw new Error(`[AAWR] Destructive command blocked. Approval ticket created.`);
      }

      // Re-run in OpenSandbox container — host is never touched
    }
  }
});
```

```

const sbResp = await fetch(`${process.env.OPENSANDBOX_URL}/sandboxes`,
  { method:"POST", body: JSON.stringify({ image:"ubuntu:22.04", ttl_seconds:120 })});
const sb = await sbResp.json();
const cmdResp = await fetch(
  `${process.env.OPENSANDBOX_URL}/sandboxes/${sb.id}/exec`,
  { method:"POST", body: JSON.stringify({ command: input.command }) });
const result = await cmdResp.json();
// Terminate sandbox
await fetch(`${process.env.OPENSANDBOX_URL}/sandboxes/${sb.id}`, {method:"DELETE"});
return result.output; // return sandbox output to OpenCode LLM
},

// Hook C: Inject enterprise context during compaction
"experimental.session.compacting": async ({ session, prompt }) => {
  const ctx = await fetch(
    `${process.env.LANGFLOW_URL}/api/v1/run/${process.env.CONTEXT_FLOW_ID}`,
    { method:"POST", headers:{"x-api-key": process.env.LANGFLOW_API_KEY!,
      "Content-Type":"application/json"},
      body: JSON.stringify({ input_value: session.projectPath })
    }).then(r => r.json());
  const ectx = ctx.outputs[0].outputs[0].results.message.text;
  return `${prompt}\n\n<enterprise_context>\n${ectx}\n\n</enterprise_context>`;
},

// Hook D: Terraform file watcher → infra validation → security audit
"file.edited": async ({ file, session }) => {
  if (!file.path.endsWith(".infra.tf")) return;
  const valid = await fetch(
    `${process.env.LANGFLOW_URL}/api/v1/run/${process.env.INFRA_FLOW_ID}`,
    { method:"POST", body: JSON.stringify({ input_value: file.path }) })
    .then(r => r.json()).then(d => d.outputs[0].outputs[0].results.message.text === "VALID");
  if (valid) await fetch(`${process.env.DEEPAGENTS_MCP_URL}/spawn`,
    { method:"POST", body: JSON.stringify({ task: `Security audit ${file.path}`,
      memory_namespace: "security_audit", context_files: [file.path] })});
}
},

// Custom tool: spawn Deep Agent subagent from within OpenCode LLM
tools: [
  tool({ name: "deep_agent_task",
    description: "Spawn a Deep Agent for long-horizon planning, RAG, or cross-session memory tasks",
    parameters: z.object({
      task_description: z.string(),
      context_files: z.array(z.string()).optional(),
      memory_namespace: z.string().optional()
    }),
    execute: async ({ task_description, context_files, memory_namespace }) => {
      const r = await fetch(`${process.env.DEEPAGENTS_MCP_URL}/spawn`,
        { method:"POST", body: JSON.stringify(
          { task: task_description, context_files, memory_namespace: memory_namespace??"default"
        })});
      return JSON.stringify((await r.json()).output, null, 2);
    }
  })
]
});

```

7. Langflow Flow: Auto App Development (Q5)

Ten component nodes forming the end-to-end Auto App Development pipeline. OpenSandbox is used at node 8 to run tests in isolation. Each node definition includes type, parameters, and connections.

Node	Type	Operation	Connects To
N1	WebhookInput	Input: GitHub issue URL	→ N2
N2	MCPTool (github-mcp)	github.get_issue(url) → {title, body, labels}	→ N3
N3	LLMComponent (gpt-4o)	Classify: bug / feature / refactor based on labels + title	→ N4 (feature) or N5
N4	RAGComponent	Query: SKILL.md vector store + architecture-decisions → technical spec	→ N5
N5	HTTPRequest	POST /session {projectPath, agentId:'feature-dev'} → sessionId	→ N6
N6	HTTPRequest	POST /session/{id}/message {content: spec + instructions}	→ N7
N7	SSEListenerComponent	GET /event — stream until session.idle fires	→ N8
N8a	HTTPRequest	GET /session/{id}/diff → FileDiff[]	→ N8b
N8b	MCPTool (opensandbox-mcp)	sandbox_create {image:'node:20'} + sandbox_exec_command 'npm test'	→ N9
N9	MCPTool (deepagents-mcp)	deepagent_spawn {task:'write unit tests', context: diffs}	→ N10
N10	MCPTool (github-mcp)	github.create_pull_request {branch, title, body: summary + test results}	→ Output

Key API Request/Response Bodies

N5: Create OpenCode session

```
# POST http://opencode-service:4096/session
# Authorization: Basic ${BASE64_CREDS}
{ "projectPath": "/repos/target-service", "agentId": "feature-dev", "share": false }
# → { "id": "sess_abc123", "status": "running", "createdAt": "..." }
```

N8b: Create sandbox + run tests

```
# POST http://opensandbox-service:8080/sandboxes
{ "image": "node:20-alpine", "cpu": 2, "memory": "1g", "ttl_seconds": 300 }
# → { "id": "sb_xyz789", "status": "running", "endpoints": {...} }

# POST http://opensandbox-service:8080/sandboxes/sb_xyz789/execd/command
{ "command": "npm ci && npm test -- --reporter json", "working_dir": "/workspace" }
# → SSE stream: { "type": "stdout", "data": "...", ... }, ..., { "type": "exit", "code": 0 }
```

8. Deep Agents as the Planning Brain (Q6)

The Deep Agent is initialized with five custom tools that give it full reach across the AAWR stack. LangGraph Memory Store accumulates enterprise knowledge across sessions. Subagent spawning parallelizes work across microservices.

Tool Name	Purpose	Implementation
create_opencode_session	Delegate coding tasks to OpenCode	POST /session + POST /session/{id}/message via httpx
trigger_langflow_flow	Trigger orchestration flows (CI, approval, RAG, POS Ops)	POST /api/v1/run/{flow_id} with x-api-key header
lsp_navigate	Navigate codebase via LSP before issuing code	POST /session/{id}/message with lsp tool call
create_opensandbox_session	Spin up isolated sandbox for test execution or POS	POST /sandboxes via OpenSandbox REST API
run_code_in_sandbox	Execute code in stateful CodeInterpreter (profiling, analysis)	POST /sandboxes/{id}/execd/code with language + context_id

aawr/planning_brain.py — Core Tools

```
from langchain_deepagents import create_deep_agent
from langchain.tools import tool
from langgraph.store.memory import InMemoryStore
import httpx, os, asyncio

OPENCODE_URL = os.environ["OPENCODE_URL"]
OPENCODE_CREDS = os.environ["OPENCODE_BASIC_AUTH"]
LANGFLOW_URL = os.environ["LANGFLOW_URL"]
OPENSANDBOX_URL = os.environ["OPENSANDBOX_URL"]

@tool
def create_opencode_session(project_path: str, prompt: str, agent_id: str = "default") -> dict:
    """Create an OpenCode session and submit a coding task. Returns session ID."""
    with httpx.Client(base_url=OPENCODE_URL,
                      headers={"Authorization": f"Basic {OPENCODE_CREDS}"}) as c:
        sess = c.post("/session", json={"projectPath": project_path, "agentId": agent_id})
        sid = sess.json()["id"]
        c.post(f"/session/{sid}/message", json={"content": prompt, "role": "user"})
        return {"session_id": sid, "status": "submitted"}

@tool
def trigger_langflow_flow(flow_id: str, input_value: str, tweaks: dict = {}) -> str:
    """Trigger a Langflow flow and return the result synchronously."""
    resp = httpx.post(f"{LANGFLOW_URL}/api/v1/run/{flow_id}",
                     headers={"x-api-key": os.environ["LANGFLOW_API_KEY"]},
                     json={"input_value": input_value, "tweaks": tweaks, "output_type": "chat"})
    return resp.json()["outputs"][0]["outputs"][0]["results"][0]["message"]["text"]

@tool
def run_code_in_sandbox(language: str, code: str, sandbox_id: str = None) -> dict:
    """Execute code in an isolated OpenSandbox CodeInterpreter session.
    Maintains state across calls if sandbox_id is reused (stateful Jupyter-backed)."""
    with httpx.Client(base_url=OPENSANDBOX_URL,
                     headers={"Authorization": f"Bearer {os.environ['OPENSANDBOX_API_KEY']}"}) as c:
        if not sandbox_id:
            sb = c.post("/sandboxes", json={"image": "python:3.11", "ttl_seconds": 600})
            sandbox_id = sb.json()["id"]
            result = c.post(f"/sandboxes/{sandbox_id}/execd/code",
                           json={"language": language, "code": code, "stream": False})
            return {"sandbox_id": sandbox_id, "output": result.json()}
        memory_store = InMemoryStore() # replace with LangGraph server store in production
```

```
SYSTEM_PROMPT = """
You are the AAWR Planning Brain.
Tools: create_opencode_session | trigger_langflow_flow | run_code_in_sandbox
       lsp_navigate | write_todos | spawn_subagent
RULES:
1. Use lsp_navigate to understand codebase before instructing OpenCode.
2. Use write_todos at the start of every multi-step task.
3. For destructive/irreversible actions trigger langflow approval flow first.
4. Store learnings in memory namespace 'enterprise_kb'.
5. Use run_code_in_sandbox for all test execution, profiling, and benchmarking.
6. Use parallel subagents for independent microservices.
"""

planning_brain = create_deep_agent(
    tools=[create_opencode_session, trigger_langflow_flow, run_code_in_sandbox],
    system_prompt=SYSTEM_PROMPT,
    store=memory_store,
    checkpointer=True,
    streaming=True,
)
```


9. SKILL.md Living Knowledge System (Q7)

SKILL.md files are reusable instruction sets stored in `.opencode/skills/`. They are loaded on-demand by agents via the built-in skill tool. The AAWR uses them as the enterprise brain — encoding conventions, policies, and learned mitigations.

File	Agent Access	Contents	Auto-updated By
git-conventions.md	All agents	Branch naming, commit format, PR template, squash merge parallelizing skills	skill-updater
deployment-process.md	devops-bot only	Staging→canary→prod flow, rollback triggers, healthcheck validation, OpenSandbox test gate	OpenSandbox
testing-standards.md	feature-dev, incident-responder	Unit/integration/e2e ratios, coverage thresholds per language, OpenSandbox image selection	OpenSandbox
security-policy.md	security-auditor only	CVE SLA, secret management, bash deny-list patterns, OpenSandbox Visor config	OpenSandbox
incident-response-playbook.md	incident-responder (read/write)	Severity (read), RCA steps, fix branch convention, Post-mortem Deep Agent, OpenSandbox validation	Deep Agent
architecture-decisions.md	All agents (read)	Active ADRs, deprecated patterns, preferred libraries, OpenSandbox skill-updater	skill-updater
opensandbox-images.md	feature-dev, devops-bot	Canonical sandbox images per language/use-case, OpenSandbox recommendations, TTL settings	OpenSandbox

incident-response-playbook.md (excerpt)

```
---
name: incident-response-playbook
description: AAWR automated incident response runbook with OpenSandbox validation steps
license: internal
compatibility: ["opencode>=0.1", "deepagents>=0.1", "opensandbox>=1.0"]
metadata:
  owner: sre-team
  last_updated_by: aawr-auto
  version: "3.2"
---

## Severity Classification
- SEV1: p99 > 2s or error_rate > 5% – immediate auto-remediation
- SEV2: p99 > 500ms or error_rate > 1% – auto-remediation with human notify

## Fix Validation (OpenSandbox)
1. POST /sandboxes { image: "repo-base:latest", cpu: 4 } → sandbox_id
2. Upload all changed files via POST /sandboxes/{id}/execd/files/upload
3. POST /sandboxes/{id}/execd/command { command: "make test-integration" }
4. Stream SSE output until exit event – assert exit code 0
5. Retrieve metrics: GET /sandboxes/{id}/execd/metrics
6. Destroy sandbox: DELETE /sandboxes/{id}

## Post-Mortem Template
- Root cause: [Deep Agent RCA output]
- Blast radius: [affected services from lsp.incomingCalls]
- Fix applied: [commit SHA]
- Test evidence: [OpenSandbox run ID + exit code + coverage delta]
- Metrics before/after: [Observability MCP snapshot]
```

10. Auto Reactive Observability Pipeline (Q8)

Five-step autonomous incident response pipeline. OpenSandbox is inserted at Step 3 to run the test suite in a fully isolated container, preventing any risk of contaminating the fix branch or production environment during validation.

Step 1	Langflow Triage	PagerDuty/Sentry webhook → classify severity (SEV1/2/3) → RAG query of past incidents → select SKILL.md remediation playbook → dispatch to Deep Agent with full context.
Step 2	Deep Agent RCA	Query LangGraph Memory Store for past fixes to this service. Use OpenCode LSP incomingCalls + findReferences to trace error call chain. Use grep + glob to identify all affected files. Produce RCA report.
Step 3	OpenCode + OpenSandbox	opencode-worktree creates fix branch. OpenCode implements fix with LSP feedback loop. OpenSandbox : create sandbox with repo image, upload changed files, run test suite via sandbox_exec_command, stream SSE output, assert exit 0. GitHub MCP opens PR with RCA + test evidence.
Step 4	Langflow Validation Gate	DevOps MCP deploys to staging. Observability MCP monitors for 5 minutes. If metrics nominal → auto-approve PR. If degraded → Slack escalation to human engineer.
Step 5	Deep Agent Post-Mortem	Generate structured post-mortem (root cause, fix, metrics before/after, OpenSandbox run ID). Update incident-response-playbook.md SKILL.md. Post to Confluence + Slack via MCP.

Step 3 — OpenSandbox Test Execution API Calls

OpenSandbox validation during incident fix

```
# 1. Create sandbox with production-like image
POST http://opensandbox-service:8080/sandboxes
Authorization: Bearer ${OPENSANDBOX_API_KEY}
{ "image": "company/service-base:latest", "cpu": 4,
  "memory": "2g", "ttl_seconds": 600,
  "metadata": { "incident_id": "INC-4521", "branch": "fix/incident-..." } }
# → { "id": "sb_inc4521", "status": "running" }

# 2. Upload changed files
POST /sandboxes/sb_inc4521/execd/files/upload
[multipart: changed_files from OpenCode diff]

# 3. Run test suite – streaming SSE
POST /sandboxes/sb_inc4521/execd/command
{ "command": "make test && make test-integration", "working_dir": "/workspace" }
# SSE → { "type":"stdout","data":"..." } ... { "type":"exit","code":0 }

# 4. Get metrics snapshot
GET /sandboxes/sb_inc4521/execd/metrics
# → { "cpu_pct": 34.2, "mem_mb": 412, "duration_ms": 18450 }

# 5. Destroy sandbox
DELETE /sandboxes/sb_inc4521
```

11. Multi-Agent Coordination Patterns (Q9)

11.1 Fork-Join Pattern

The Deep Agent planning brain spawns one OpenCode session per microservice in parallel (`asyncio.gather`). Each session runs in its own OpenSandbox container for test isolation. The join phase collects all `FileDiff[]` results and produces a single consolidated PR.

fork_join.py

```
import asyncio

async def fork_join_pipeline(services: list[dict], task: str):
    async def run_service(svc):
        # Fork: create OpenCode session
        sess = await create_opencode_session.ainvoke({
            "project_path": svc["path"], "prompt": task, "agent_id": svc["agent_id"]
        })
        await wait_for_session_idle(sess["session_id"])
        diffs = await get_session_diff(sess["session_id"])

        # Test in isolated sandbox per service
        sb = await create_sandbox(image=svc["image"], cpu=2, memory="1g")
        test_result = await run_tests_in_sandbox(sb["id"], diffs)
        await destroy_sandbox(sb["id"])
        return {"service": svc["name"], "diffs": diffs, "tests": test_result}

    # All services in parallel
    results = await asyncio.gather(*[run_service(s) for s in services])

    # Join: consolidate PR via planning brain
    consolidated = planning_brain.invoke({"messages": [{"role": "user",
        "content": f"Merge these service results into a consolidated PR:\n{results}"}]})
    await trigger_langflow_flow.ainvoke({
        "flow_id": os.environ["CREATE_PR_FLOW_ID"], "input_value": consolidated["output"]})
```

11.2 Supervisor-Worker Pattern

Langflow supervisor flow dispatches tasks to a pool of OpenCode workers. Workers report status via SSE. When a worker becomes idle, the supervisor rebalances the queue and re-assigns.

supervisor_worker.ts

```
async function supervisorRebalanceLoop(
    workerPool: string[], taskQueue: string[], client: OpenCodeClient
) {
    const status = new Map<string, "busy"|"idle">(workerPool.map(id => [id, "idle"]));

    while (taskQueue.length > 0) {
        for (const [wid, st] of status.entries()) {
            if (st === "idle" && taskQueue.length > 0) {
                const task = taskQueue.shift();
                await client.session.prompt(wid, { content: task, role: "user" });
                status.set(wid, "busy");
                const stream = client.events.subscribe({ sessionId: wid });
                stream.on("session.idle", () => status.set(wid, "idle"));
                stream.on("session.error", () => { status.set(wid, "idle"); taskQueue.push(task); });
            }
        }
        await new Promise(r => setTimeout(r, 2000));
    }
}
```

```
}  
}
```

11.3 Reactive Event Chain (Terraform → Infra Validation → Security Audit)

An OpenCode plugin listens for `file.edited` on `*.infra.tf` files. On match: trigger Langflow infra validation → if valid, spawn Deep Agent security audit in OpenSandbox.

Reactive chain — hook in `aawr-deepagents-bridge.ts`

```
"file.edited": async ({ file }) => {  
  if (!file.path.endsWith(".infra.tf")) return;  
  
  // Step 1: Langflow infra validation (terraform plan + tflint)  
  const validResp = await fetch(`${LF}/api/v1/run/${INFRA_FLOW_ID}`,  
    { method: "POST", body: JSON.stringify({ input_value: file.path }) }).then(r=>r.json());  
  const isValid = validResp.outputs[0].outputs[0].results.message.text === "VALID";  
  if (!isValid) { /* notify engineer via Slack MCP */ return; }  
  
  // Step 2: Deep Agent security audit in OpenSandbox  
  const sb = await fetch(`${OS}/sandboxes`,  
    { method: "POST", body: JSON.stringify({ image: "hashicorp/terraform:latest", ttl_seconds: 300 }) })  
  .then(r=>r.json());  
  await fetch(`${DAGENTS}/spawn`, { method: "POST", body: JSON.stringify({  
    task: `Run tfsec + checkov on ${file.path}. Produce findings report.`,  
    memory_namespace: "security_audit", sandbox_id: sb.id }) });  
}
```

12. Enterprise Deployment Architecture (Q10)

OpenCode Fleet

Run one opencode serve container per project/repo behind a per-project router. Use Kubernetes Deployment with readinessProbe on GET /ping. Route by hostname: opencode-{repo}.internal → pod. Register each server with Langflow service registry on startup.

Remote Config Policy

Host /.well-known/opencode on enterprise gateway. Global policy: bash: ask, write: ask, github-mcp_delete_*: deny. Production-repo override: bash: deny, write: deny. This config is merged before project opencode.json.

Langflow Namespaces

Deploy Langflow behind an API gateway (Kong/Traefik). Enforce per-team flow namespaces via JWT-scoped route policies. Per-team MCP tool RBAC: team-alpha can call github-mcp_*; platform-sre can call devops-mcp_* + observability-mcp_*.

Deep Agents Scaling

Run LangGraph Server cluster behind load balancer. All nodes share a Postgres-backed LangGraph Memory Store for cross-team knowledge. Use checkpointer=True with a shared checkpoint store for durable execution across restarts.

OpenSandbox Fleet

Deploy OpenSandbox server with Kubernetes BatchSandbox runtime for pooling. Pre-warm 20 Python + 10 Node.js + 5 Go sandbox instances per cluster. Use kata-containers or gVisor for production security hardening.

Observability Stack

LangSmith: all Deep Agent traces with full tool call logs. opencode-sentry-monitor plugin: all OpenCode session events → Sentry. OpenSandbox /metrics/watch SSE: sandbox CPU/memory → Datadog. Langflow execution logs → Grafana. Unified dashboard: every autonomous action is traceable.

Circuit Breaker

Track per-agent: failed_attempts ≥ 3 OR total_cost_usd ≥ \$50 → trip breaker. Add OpenSandbox CPU quota as a second budget axis: sandbox_cpu_hours > threshold → trip. On trip: pause execution, notify #aawr-alerts Slack, archive LangSmith trace for human review. Auto-reset after 30-minute cooldown.

Secret Management

All secrets in HashiCorp Vault or AWS Secrets Manager. Injected via OpenCode shell.env plugin hook at runtime. Use OPENCODE_CONFIG_CONTENT env var pattern for runtime config override. Never write secrets to opencode.json or SKILL.md files.

Remote Config: Enterprise Security Policy

```
https://enterprise.com/.well-known/opencode
```

```
{
  "permissions": {
    "bash": "ask",
```

```
    "write": "ask",
    "opensandbox-mcp_sandbox_create": "allow",
    "opensandbox-mcp_*": "allow",
    "github-mcp_create_*": "allow",
    "github-mcp_merge_*": "ask",
    "github-mcp_delete_*": "deny",
    "devops-mcp_deploy_production": "ask"
  },
  "$prod-repos": {
    "permissions": {
      "bash": "deny",
      "write": "deny",
      "github-mcp_merge_*": "ask",
      "devops-mcp_deploy_*": "ask"
    }
  }
}
```

13. Reference Implementation Blueprint (Q11)

File/Folder Structure

```

aawr/
├── docker-compose.yml
├── k8s/
│   ├── opencode-deployment.yaml
│   ├── langflow-deployment.yaml
│   ├── langgraph-server-deployment.yaml
│   ├── opensandbox-deployment.yaml    # NEW — BatchSandbox Kubernetes runtime
│   └── ingress.yaml
├── .github/workflows/bootstrap-aawr.yml
├──
├── opencode/
│   ├── opencode.json                  # MCP servers + permissions config
│   ├── AGENTS.md                     # custom agent definitions
│   └── .opencode/
│       ├── skills/
│       │   ├── git-conventions.md
│       │   ├── deployment-process.md
│       │   ├── testing-standards.md
│       │   ├── security-policy.md
│       │   ├── incident-response-playbook.md
│       │   ├── architecture-decisions.md
│       │   └── opensandbox-images.md  # NEW — canonical images per language
│       └── plugins/
│           ├── aawr-deepagents-bridge.ts
│           ├── aawr-circuit-breaker.ts
│           └── aawr-infra-watcher.ts
│       └──
├── langflow/
│   ├── flows/
│   │   ├── auto-app-development.json
│   │   ├── incident-triage.json
│   │   ├── staging-validation.json
│   │   ├── human-approval-gate.json
│   │   └── skill-updater.json
│   └── mcp_server_config.json
│   └──
├── deepagents/
│   ├── planning_brain.py
│   ├── tools/
│   │   ├── opencode_tools.py
│   │   ├── langflow_tools.py
│   │   ├── lsp_tools.py
│   │   └── opensandbox_tools.py      # NEW — sandbox_create, run_code_in_sandbox
│   ├── memory/enterprise_kb.py
│   └── mcp_server.py
│   └──
├── opensandbox/                      # NEW
│   ├── images/
│   │   ├── python-base.Dockerfile
│   │   ├── node-base.Dockerfile
│   │   └── go-base.Dockerfile
│   └── mcp_adapter/
│       └── opensandbox_mcp_server.ts
│       └──
├── mcp_servers/
│   ├── github/
│   ├── devops/
│   └── observability/

```

■■■ confluence/

AGENTS.md

opencode/AGENTS.md

AAWR Custom Agents

feature-dev

Skills: git-conventions, testing-standards, architecture-decisions, opensandbox-images

Permissions:

bash: ask

write: allow

opensandbox-mcp_*: allow # can create sandboxes for test execution

github-mcp_create_pull_request: allow

github-mcp_merge_*: deny

security-auditor

Skills: security-policy

Permissions:

bash: deny

write: ask

opensandbox-mcp_sandbox_create: allow # for isolated security scans

security-mcp_*: allow

incident-responder

Skills: incident-response-playbook, deployment-process

Permissions:

bash: ask

write: allow

opensandbox-mcp_*: allow # test suite execution in sandbox

github-mcp_*: allow

devops-mcp_rollback: allow

perf-optimizer

Skills: testing-standards, architecture-decisions, opensandbox-images

Permissions:

bash: deny

opensandbox-mcp_*: allow # all profiling + benchmarking in sandbox

write: allow

14. Failure Modes, Risks & Mitigations (Q12)

Top 10 failure scenarios with detection signals, automated mitigations, and human escalation paths. OpenSandbox mitigates failures #3, #4, and #8 directly — these were previously risks in the three-tool baseline architecture.

#	Failure	Detection	Auto Mitigation	Escalation
1	Infinite loop — agent retries failing tool call indefinitely	Circuit breaker: attempts > 3; LangSmith trace depth > threshold	CB trips; Deep Agent HITL interrupt; session archived	Slack alert; SRE reviews LangSmith trace
2	Token budget explosion — LLM generates excessive context	total_cost_usd > \$50 in circuit breaker	Session aborted; opencode-dyn amic-context-pruning activates	Cost alert to team lead; session archived
3	Host contamination — AI-generated code damages host filesystem	<i>Prevented by OpenSandbox</i> — all code execution in isolated container	OpenSandbox container TTL expires; host is never touched	N/A — architecture eliminates this risk
4	Destructive bash command — rm -rf, kubectl delete on host	tool.execute.before hook matches deny-list pattern	Routed to Langflow approval gate; if approved, runs in OpenSandbox not host	Human receives approval request with full command + context
5	LSP server crash — no language feedback to LLM	lsp.diagnostics event stops; session continues silently	Session flagged; Deep Agent switches to grep/glob fallback mode	Engineer notified; LSP server restarted via DevOps MCP
6	Merge conflict storm — parallel agents edit same file	session.diff returns overlapping hunks at join phase	Fork-join paused; Deep Agent consolidates sequentially	Engineer reviews final merge via draft PR
7	Staging metrics degraded post-deploy	Observability MCP returns error_rate > threshold after 5-min window	Auto-rollback via DevOps MCP; PR blocked from merging	PagerDuty alert; incident-triage flow activated
8	Secret leakage in agent output	Regex scan on session output for key/token patterns; OpenSandbox env vars isolated	Output sanitized; LangSmith trace redacted; OpenSandbox env never leaked	Security team alerted; credential rotated via Vault
9	MCP server unavailable — tool calls fail silently	mcp_tool_result returns error; retry count exceeded	Langflow marks subtask blocked; retries with exponential backoff	Engineer alerted to restore MCP server
10	Compaction destroys critical context	Subsequent tool calls contradict earlier decisions in LangSmith diff	Hook injects enterprise context; Deep Agent HITL if contradiction detected	Human reviews session history

15. Implementation Roadmap

Phase 1 — Foundation (Weeks 1–6)

- OpenCode fleet setup (Docker/K8s) — one server per repo
- Langflow shared service deployment with team namespaces
- MCP Protocol Bridge: Langflow ↔ OpenCode ↔ Deep Agents
- OpenSandbox server deployment with Docker runtime
- Initial SKILL.md knowledge base (6 core files)
- Basic AGENTS.md + permission config per team
- Unlocks: basic MCP bus, LSP-aware code execution, isolated test sandboxes

Phase 2 — Autonomy (Weeks 7–16)

- Auto App Development pipeline (GitHub Issue → PR)
- Auto DevOps pipeline (merge → test → deploy → rollback)
- Auto Reactive Observability pipeline (alert → fix → PR → post-mortem)
- OpenCode plugin: aawr-deepagents-bridge (all 4 hooks)
- Fork-Join + Supervisor-Worker coordination patterns
- OpenSandbox BatchSandbox K8s runtime with sandbox pooling
- Unlocks: end-to-end autonomous delivery, reactive incident response

Phase 3 — Enterprise Hardening (Weeks 17–28)

- Circuit breaker + token budget + sandbox CPU quota controls
- Secret management (HashiCorp Vault integration)
- Remote config (.well-known/opencode) enterprise policies
- LangSmith + Sentry + Datadog full observability stack
- Auto Security Patching pipeline (CVE → patch → scan → PR)
- Auto Performance Optimization (regression → profiling → benchmark loop)
- Auto Knowledge Base Maintenance (PR/ADR → SKILL.md update)
- Multi-region deployment + mDNS discovery + OpenSandbox gVisor
- Unlocks: all 6 enterprise use-cases at production grade

16. Capability Gaps & Open Questions

All workarounds use only documented APIs. Gaps are identified where point-to-point integration is required because native support is absent in the current tool versions.

Gap	Affected Use-Case	Workaround	Future Feature Needed
OpenCode has no native session queue	Supervisor-Worker at scale	One OpenCode server per project/repo behind a per-project router	OpenCode session concurrency controls / queue
Langflow has no native SSE listener component	All pipelines polling session progress	Custom Python component with <code>htpx.stream()</code> to consume SSE	Langflow built-in SSE listener node
Deep Agents SDK has no native MCP server hosting	MCP Protocol Bridge	Run <code>deepagents_mcp_server.py</code> as a sidecar via <code>stdio/SSE</code>	Native MCP server mode in LangChain Deep Agents
OpenCode <code>.well-known/opencode</code> lacks per-user RBAC	Multi-tenant enterprise fleet	Per-repo <code>opencode.json</code> agent overrides as proxy for per-user perms	OpenCode identity-aware permission resolution
LangGraph Memory Store has no TTL or namespace ACL	Cross-team knowledge isolation	Namespace keys by team prefix; TTL in application layer	LangGraph Memory Store TTL + ACL support
<code>experimental.session.compacting</code> is unstable API	All long-running sessions	Pin OpenCode version; add CI integration test for hook signature	Stable <code>session.compacting</code> hook API
OpenSandbox has no native MCP adapter	All sandbox execution via MCP bus	<code>opensandbox_mcp_server.ts</code> adapter wrapping REST API	Official OpenSandbox MCP server package
Langflow flow RBAC not enforced at MCP layer	Multi-team Langflow deployment	API gateway (Kong/Traefik) with JWT-scoped route policies	Langflow native MCP tool-level RBAC

Document Information

Version	1.1.0 — includes OpenSandbox integration layer
Stack	LangChain Deep Agents SDK · OpenCode · Langflow · OpenSandbox
Protocols	MCP · ACP · HTTP/REST · SSE · JSON-RPC · LSP
Languages	TypeScript · Python · Go · Java · Rust · Terraform
Constraints	Tool Fidelity · Enterprise Security · Human Override · Observability First · Idempotency · Polyglot
Audience	Senior distributed systems engineers, AI infrastructure teams, platform SREs